This handout includes space for every question that requires a written response. Please feel free to use it to hand write your solutions (legibly, please). If you choose to typeset your solutions, the `README.md` for this assignment includes instructions to regenerate this handout with your typeset LaTeX solutions.

3.a

Consider the case with $\lambda F = 0.99$ and $\lambda R = 0.01$. Based on the train/test loss curves, does parameter sharing outperform having separate models? (Recall that the purpose of this assignment was to train a model f(userID, itemID) that predicts: 1) the probability p that the user would watch the movie (Factorization Task), 2) the score r they would assign to it from 1 to 5 (Regression Task)).

Answer: No, not for these tasks and architecture. For these task weightings for factorization (0.99) and for regression (0.01), based on the following plots, parameter sharing (black) exhibits a higher training/Factorization loss (c) and nearly equivalent Joint loss and MSE compared to separate models (yellow) throughout the later training epochs; and also exhibits a lower Mean Reciprocal Rank Metric (b), where higher is better[1]. In the regression task, the test loss (a) shows no real distinction between parameter sharing (black) or not sharing (yellow)–both curves trend towards indistinguishable in the final epochs. And the joint training loss (d) shows a very slightly lower loss for the separate (yellow) versus shared model (black). Thus, separate models (yellow) performs better in the MRR and as well as shared models with these task weightings.

Extra note: When first writing this, I was a little concerned that training our model hadn't fully converged as indicated by the non-zero slope in the factorization training loss plot near epoch 200, so I doubled the number of training epochs–indicating more training is beneficial as long as the solution doesn't become "overfit" with too many epochs.

Additional Context: Here, λF refers to factorization weight and λR regression weight respectively.

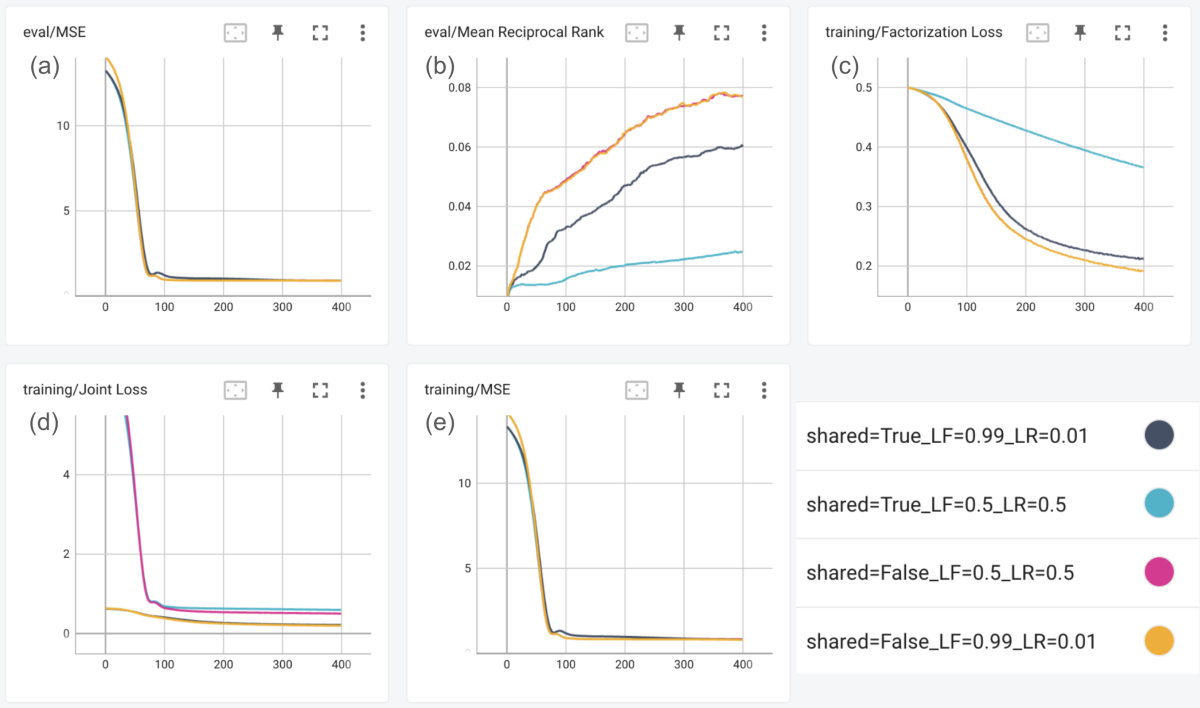


Figure: Testing (a,b) and Training (c,d,e) metrics colored as follows: Black: "shared=True_LF=0.99_LR=0.01", Blue: "shared=True_LF=0.5_LR=0.5" Red: "shared=False_LF=0.5_LR=0.5" Yellow: "shared=False_LF=0.99_LR=0.01".

[1] https://en.wikipedia.org/wiki/Mean_reciprocal_rank

3.b

Now consider the case with $\lambda F = 0.5$, and $\lambda R = 0.5$ Based on the train/test loss curves, does parameter sharing outperform having separate models?

Answer: Answer: No. For these equal task weightings for factorization (0.5) and for regression (0.5), based on the previous figure, parameter sharing (blue) exhibits a slightly higher training loss (d) throughout the later training epochs; and also exhibits a far worse/lower Mean Reciprocal Rank Metric (b), where higher is better. In the regression task, the test loss (a) shows no real distinction between parameter sharing (blue) or not sharing (red)–both curves trend towards indistinguishable in the final epochs. And the joint training loss (d) shows a marginally lower loss for the separate (red) versus shared model (blue). Thus, separate models (yellow) performs better for these task weightings (substantially so for the MRR).

3.c

In the shared model setting compare results for $\lambda F = 0.99$ and $\lambda R = 0.01$ and $\lambda F = 0.5$ and $\lambda R = 0.5$, can you explain the difference in performance?

Answer: The shared parameter paradigm benefits from a large (0.99) (black) factorization task weighting versus equal (0.5) (blue) task weighting as evidenced in the dramatic improvement in both the MRR test/eval score (b; black versus blue) and measurably lower training factorization and joint losses (c,d). Why? My theory is that too much information leakage between the regression MLP and factorization problem is detrimental to factorization performance–and we mustn't forget at least in this problem and with these experiments, separate modeling outperforms shared modeling.

Looking at plots (c) training/Factorization loss, and (e) training/MSE loss, another consideration might be due to the rates of learning convergence for these 2 tasks.

Specifically, the scoring task (via MLP regression) converges by epoch 100 while the probability that a user would watch a movie task (via Factorization) optimizes much more slowly (many more epochs needed).